

Programming SQL Server 2019

Triggers

October 26, 2019 1:01:22 PM

Prelude

You have already learnt about VIEWS, UDFs, and Stored Procedures. In this lesson we are going to learn about another special type of database object - TRIGGER. Most important feature about triggers is that you do not execute them explicitly, they run automatically in response to events. So you use them differently. Moreover, from SQL Server 2005 and on a new type of trigger is introduced. We are going to explore that too.

Go ahead and get started.

Today's topics

- What are triggers
- Common uses of triggers
- Types of triggers
- DML triggers
 - AFTER Vs. INSTEAD OF triggers
 - How AFTER triggers work
 - Creating AFTER triggers
 - Creating INSTEAD OF triggers
- DDL Triggers
 - EVENTDATA () function
 - Creating DDL triggers
- Workouts
- Practice works

Home works

- Create the tables as shown. (Choose appropriate data types for the columns)

wages table

Column name	Data Type	Key
workerid	INT	Primary Key
workername	VARCHAR(40)	
hourworked	FLOAT	
payrate	MONEY	

payments table

Column name	Data Type	Key
workerid	INT	Foreign Key
payamount	MONEY	

- Create a trigger on wages table so that whenever a record is added to the wages table, a new record is also inserted into payments table for the same workerid. The payamount column value is calculated by multiplying hourworked and payrate.

Important Points:

-
-
-
-

Summary:

-
-
-
-

What is a Trigger?

- A trigger is a special kind of parameter-less stored procedure that responds to specific events.
- Unlike Stored Procedures they are not executed explicitly, they are executed automatically in response to specific actions like inserting/updating/deleting data inside table or creating /altering database objects (tables, schema etc.).

Some common uses of triggers

- Enforcement of Referential Integrity
- Creating audit trails
- Functionality similar to a CHECK constraint, but which works across tables, databases, or even servers.
- Substituting your own statements in the place of a user's action statement (usually used to enable inserts in complex views).

Types of Trigger

- Two types
- Data Definition Language (DDL) Triggers:
They fire in response to someone changing the structure of your database in some way (CREATE, ALTER, DROP, and similar statements). These are new with SQL Server 2005.
- Data Manipulation Language (DML) Triggers:
They are bound to a table or view. They fire when we execute insert or update or delete statements for which the trigger is attached.

DML Triggers

- DML triggers can be created on a table or view
- You can attach a trigger for the following DML actions
 - Insert
 - Update
 - Delete
- A trigger can be attached to any single action mentioned above or any combination of the above actions.

Types of DML trigger

- Two types:
- After triggers:
They fire after the statement that causes it to fire, has been executed
- Instead of triggers:
They fire before the statement that cause it to fire. The statement makes the instead of triggers to fire is never executed

After Triggers

- You can attach multiple AFTER trigger per action that is, you can attach more than one AFTER triggers for INSERT or UPDATE or DELETE.
- If an action has more than one AFTER triggers, the triggers are fired as they are created. You can change the order but only you can set which one is first to fire and which one is the last.

Creating After Trigger

- Basic Syntax
- For single action

```
CREATE Trigger trigger-name
ON table-name
FOR/AFTER INSERT/UPDATE/DELETE
AS
BEGIN
    Statements
    .....
END
```
- For multiple action

```
CREATE Trigger trigger-name
ON table-name
FOR/AFTER INSERT, UPDATE, DELETE
AS
```

```
BEGIN
    Statements
    .....
END
```



Unlike procedures, triggers do not accept arguments. During a trigger is executed some temporary tables exist based on the actions on which the trigger is attached.

Workout #01

- Let's try an after trigger
- Create a table

```
CREATE TABLE tblTest
(
    C1 INT, C2 VARCHAR (5)
)
GO
```
- Insert rows into the table

```
INSERT INTO tblTest VALUES
(1, 'V1'),
(2, 'V2')
GO
```
- Create an AFTER Trigger for UPDATE, DELETE actions

```
CREATE TRIGGER trUpdateDelete
ON tblTest
AFTER UPDATE, DELETE
AS
BEGIN
    PRINT 'Trigger Fired!'
    ROLLBACK TRANSACTION
END
GO
```
- Test the trigger

```
--See the data before update/delete
SELECT * FROM tblTest
GO
UPDATE tblTest SET C2 = 'Changed Value' WHERE C1 = 1
DELETE tblTest WHERE C1 = 2
GO
--See the data after update and delete
SELECT * FROM tblTest
GO
```

Output

C1	C2
1	v1
2	v2

(2 row(s) affected)

Trigger fired!
Msg 3609, Level 16, State 1, Line 2
The transaction ended in the trigger. The...

C1	C2
1	v1
2	v2

(2 row(s) affected)

Rows are unchanged!!!- Explanation?

- Whenever an INSERT, UPDATE or DELETE statement is run, SQL Server starts an implicit transaction. The transaction is not committed until the next statement is encountered.
- In our case an AFTER trigger is attached on tblTest for UPDATE and DELETE action.
- When an UPDATE statement is run, data is updated but not committed and the trigger is fired, the code inside the trigger prints a message and rollbacks the uncommitted transaction.
- The same thing happens for the DELETE action.
- That is why records in tblTest are unchanged before and after UPDATE, DELETE statements

Workout #02

- Create the table product and stockin first

```
CREATE TABLE product
(
    pid INT PRIMARY KEY,
    pname VARCHAR(30),
    price MONEY,
    stock INT DEFAULT 0
)
GO
CREATE TABLE stockin
(
    id int IDENTITY PRIMARY KEY,
    date DATETIME DEFAULT GETDATE(),
    pid INT REFERENCES product( pid ),
    quantity INT
)
GO
```
- Insert data into product table

```
INSERT INTO product VALUES
(1, 'Mouse', 150, 0),
(2, 'Keyboard', 300, 0),
(2, 'USB Drive', 400, 0)
GO
SELECT * FROM product
GO
```
- Data in the table

pid	pname	price	stock
1	Mouse	150.00	0
2	Keyboard	300.00	0
3	USB Drive	400.00	0

- Now we want that whenever an entry is inserted in stockin table, the product table must be updated - the stock value of the product will raise by quantity value.
- We will do it using insert trigger on stockin table. For that we need to understand the after insert trigger functioning process

How an AFTER INSERT trigger works

- Suppose we have attached an AFTER INSERT trigger on Trainees table.

traineeid	traineename	email	AFTER
1108760	Asad Mostafa	asad@gmail.com	INSERT

- Now user issues an insert command
- INSERT INTO Trainees VALUES ('1109877','Jamal Uddin', 'jamal@ymail.com')
- The sequence of events occur are:
 - User issues the insert statement
 - Data is inserted into the table
 - New row is copied to a temporary table named inserted. It is available only inside the trigger
 - The insert after trigger is fired

INSERT INTO Trainees VALUES ('1109877','Jamal Uddin', 'jamal@ymail.com') ①

Trainees table

traineeid	traineename	email
1108760	Asad Mostafa	asad@gmail.com
1109877	Jamal uddin	jamal@ymail.com

Inserted temporary table

traineeid	traineename	email
1109877	Jamal uddin	jamal@ymail.com

Trigger is fired. ④

Workout #03

- Let's create our trigger

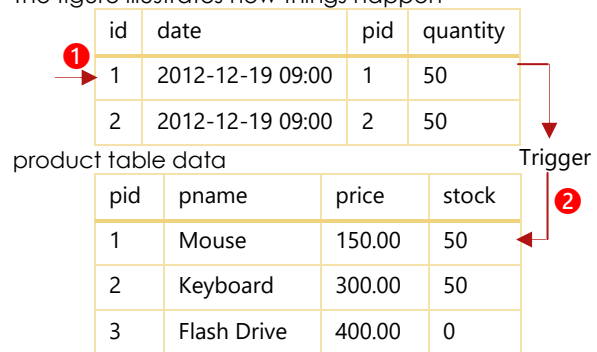
```
CREATE TRIGGER trStockIn
ON stockin
FOR INSERT
AS
BEGIN
    DECLARE @i int, -- for pid
            @q int -- for quantity

    SELECT @i=pid, @q=quantity FROM inserted

    UPDATE product
    SET stock = stock+@q
    WHERE pid = @i
END
GO
```
- Test the trigger

```
--Insert rows into stockin
INSERT INTO stockin (pid, quantity) VALUES (1, 50)
INSERT INTO stockin (pid, quantity) VALUES (2, 50)
GO
Check data in tables
SELECT * FROM stockin
SELECT * FROM product
GO
```

The figure illustrates how things happen



How things happened?

- Data insert in stockin fires the trigger
- The trigger updates the product table

How an AFTER DELETE trigger works

- Say an AFTER DELETE trigger is attached on the trainees table

traineeid	traineename	email	AFTER
1108760	Asad Mostafa	asad@gmail.com	DELETE
1109877	Jamal Uddin	jamal@ymail.com	

- When a delete command is issued, the sequence of events that occur are:
 - User issues the delete statement
 - Data is deleted from the table
 - Deleted row is copied to a temporary table named deleted. It is available only inside the trigger.
 - The after delete trigger is fired

DELETE FROM Trainees WHERE traineeid = '1109877' ①

Trainees table

traineeid	traineename	email
1108760	Asad Mostafa	asad@gmail.com
1109877	Jamal uddin	jamal@ymail.com

deleted temporary table

traineeid	traineename	email
1109877	Jamal uddin	jamal@ymail.com

The after delete trigger is fired. ④

Workout #04

- Let's create an after delete trigger on stockin table.
- When an entry is added to stockin table, the corresponding product's stock is raised
- So when an entry is deleted from stockin table, the corresponding product's stock is lowered
- Let's create it

```
CREATE TRIGGER trStockDel
ON stockin
FOR DELETE
AS
BEGIN
    DECLARE @i INT, @q INT
    SELECT @i = pid, @q=quantity FROM deleted
    UPDATE product
    SET stock = stock - @q
    WHERE pid = @i
```

END

GO

- Test the trigger
 - values before delete
 - SELECT * FROM stockin
 - SELECT * FROM product
 - GO
 - DELETE stockin WHERE id = 1
 - GO
 - values after delete and execution of trigger
 - SELECT * FROM stockin
 - SELECT * FROM product
 - GO
- Results of the SQL commands

stockin table (before delete)

id	date	pid	quantity
1	2012-12-19 09:00	1	50
2	2012-12-19 09:00	2	50

product table (after delete)

pid	pname	price	stock
1	Mouse	150.00	50
2	Keyboard	300.00	50
3	Flash Drive	400.00	0

stockin table (before delete)

id	date	pid	quantity
2	2012-12-19 09:00	2	50

product table (after delete)

pid	pname	price	stock
1	Mouse	150.00	0
2	Keyboard	300.00	50
3	Flash Drive	400.00	0

Trigger

How things happened?

- Data delete in stockin fires the trigger
- The trigger updates the product table

How an AFTER UPDATE trigger works

- Say an AFTER DELETE trigger is attached on the trainees table

traineeid	trainee name	email	AFTER
1108760	Asad Mostafa	asad@gmail.com	UPDATE
1109877	Jamal uddin	jamal@ymail.com	

Let's see how an after update trigger work when user issues the update command

UPDATE trainees SET email = 'jamal@gmail.com' WHERE traineeid = '1105877'

- The sequence of events occur are:
 - User issues the update statement
 - Original row is deleted from the table
 - Deleted row is copied to "deleted" table
 - A new row with updated value is inserted to the table
 - New row is copied to the inserted table.
 - Update trigger is fired

UPDATE trainees SET email = 'jamal@gmail.com' WHERE traineeid = '1105877' ①

trainees table

traineeid	trainee name	email
1108760	Asad Mostafa	asad@gmail.com
1109877	Jamal uddin	jamal@ymail.com
1109877	Jamal uddin	jamal@gmail.com

deleted temporary table

traineeid	trainee name	email
1109877	Jamal uddin	jamal@ymail.com

inserted temporary table

traineeid	trainee name	email
1109877	Jamal uddin	jamal@gmail.com

The update trigger is fired and code inside the trigger executes ⑥

Workout #05

- Let's create an after update trigger on stockin table
- An insert in stockin table raises the stock in product table
- If a stockin table is updated there should be a trigger to adjust the stock with change in quantity.
- The update trigger should check if quantity value is updated or not. If quantity is updated, it must adjust stock accordingly.
- Here the UPDATE function is used to check if the quantity column is updated or not.

CREATE TRIGGER trStockUpdate

ON stockin

AFTER UPDATE

AS

BEGIN

IF UPDATE (quantity)

BEGIN

DECLARE @i INT, @oq INT,

@nq INT, @change INT

SELECT @i = i.pid,

@oq = d.quantity, @nq = i.quantity

FROM inserted i

INNER JOIN deleted d

ON i.id = d.id

SET @change = @nq - @oq

UPDATE product SET

stock = stock - @ change

WHERE pid = @i

END

END

GO

- Test the trigger

-- values before update

SELECT * FROM stockin

SELECT * FROM product

GO

UPDATE stockin SET quantity = 80

```

WHERE id = 2
GO
-- Values after update and
-- execution of trigger
SELECT * FROM stockin
SELECT * FROM product
GO

```

- Results of SQL commands
stockin table (before update)

id	date	pid	quantity
2	2012-12-19 09:00	2	50

product table (before update)

pid	pname	price	stock
1	Mouse	150.00	0
2	Keyboard	300.00	50
3	Flash Drive	400.00	0

stockin table (after update)

id	date	pid	quantity
2	2012-12-19 09:00	2	80

product table (after update)

pid	pname	price	stock
1	Mouse	150.00	0
2	Keyboard	300.00	80
3	Flash Drive	400.00	0



How things happened?

- Data update in stockin fires the trigger
- The trigger updates the product table

i

UPDATE function returns a Boolean value that indicates whether an INSERT or UPDATE attempt was made on a specified column of a table or view.

Instead of triggers

- Instead of trigger fires before the execution of the statement that cause it to fire
- You can attach only one instead of trigger per action - one for insert, one for update, one for delete
- The statement, that causes the instead of trigger to fire, never executes
- After trigger can only be attached to tables but instead of triggers can be attached to both tables and views.

Uses of Instead of Triggers

- The primary advantage of INSTEAD OF triggers is that they enable views, which would not be updatable, to support updates.
- A view based on multiple base tables must use an INSTEAD OF trigger to support inserts, updates, and deletes

Workout #06

- Let's try an instead of trigger
- Create the sales table
CREATE TABLE sales
(
id INT IDENTITY PRIMARY KEY,
product VARCHAR(30),
price MONEY,
quantity INT,
amount AS price * quantity
)
GO

- Insert a row
INSERT INTO sales (product, price, quantity)
VALUES ('Mouse', 250, 10)
GO
Create a view
CREATE VIEW vsales
AS
SELECT id, product, price, quantity, amount
FROM sales
GO
SELECT * FROM vsales
GO
- Rows returned by the View

product	price	quantity	amount
Mouse	250.00	10	2500.00

- We want to insert data through the vsales
- One problem in the view is that we have to give explicit values for all columns in the view
- But the amount column in sales is calculated. It does not accept explicit values.
- We will solve this using an instead of insert trigger
- Create the view
CREATE TRIGGER trvsalein
ON vsales
INSTEAD OF INSERT
AS
BEGIN
--insert only required ones
--ignore unwanted values
INSERT INTO sales
(product, price, quantity)
SELECT product, price, quantity
FROM inserted
END
GO

- Test the trigger to see if the instead of trigger works or not.
- We want to insert data through the vsales
--values before insert
SELECT * FROM sales
GO
INSERT INTO vsales VALUES (999, 'Keyboard', 250, 2, 0)
--fake values are given for id and amount
--these values will be ignored by the trigger
GO
SELECT * FROM sales
GO

- Rows in tables
Data in sales table (before insert)

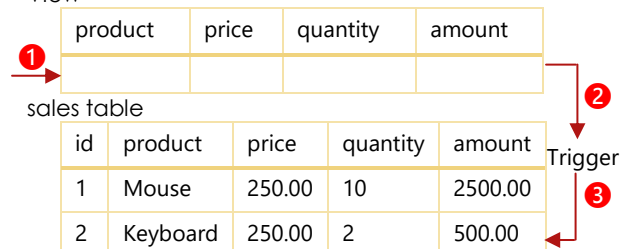
id	product	price	quantity	amount
1	Mouse	250.00	10	2500.00

Data in sales table (after insert)

id	product	price	quantity	amount
1	Mouse	250.00	10	2500.00
2	Keyboard	250.00	2	500.00

How things happened?

View



1. Insert command on the view is issued
2. Insert command on the view fires the trigger
3. The trigger inserts the row into the table

Disabling DML Triggers

- You can disable all triggers or a particular trigger attached to a table without dropping
- Syntax to Disable all trigger
ALTER TABLE table-name
DISABLE TRIGGER ALL
- Disable a particular trigger
ALTER TABLE table-name
DISABLE TRIGGER trigger-name

Enabling DML Triggers

- You can enable a disabled trigger or all disabled triggers attached to a table
- Syntax to Enable all trigger
ALTER TABLE table-name
ENABLE TRIGGER ALL
- Enable a particular trigger
ALTER TABLE table-name
ENABLE TRIGGER trigger-name

DDL Triggers

- DDL triggers new from SQL Server 2005.
- They fire in response to someone changing the structure of your database in some way (CREATE, ALTER, DROP, and similar statements).
- DDL triggers, like standard triggers, execute in response to an event.
- But unlike standard triggers, they do not execute in response to UPDATE, INSERT, or DELETE statements on a table or view.
- Instead, they primarily execute in response to data definition language (DDL) statements. These include CREATE, ALTER, DROP, GRANT, DENY, REVOKE, and UPDATE STATISTICS statements.

EVENTDATA () function

- It returns information about server or database events
- EVENTDATA returns data only when referenced directly inside of a DDL or logon trigger
- EVENTDATA returns XML data. This data is sent to the client as Unicode
- EVENTDATA function returns data as below
<EVENT_INSTANCE>
<EventType>type</EventType>
<PostTime>date-time</PostTime>
<SPID>spid</SPID>
<ServerName>name</ServerName>
<LoginName>name</LoginName>
<UserName>name</UserName>
<DatabaseName>Name</DatabaseName>
<SchemaName>name</SchemaName>
<ObjectName>name</ObjectName>
<ObjectType>type</ObjectType>
<TSQLCommand>command</TSQLCommand>
</EVENT_INSTANCE>
- DDL triggers can be created DATABASE or SERVER (Database-scoped or Server-coped)
- Using a server-scoped DDL trigger statements that can be captured are

	ALTER_AUTHORIZATION_SERVER	
CREATE_DATABASE	ALTER_DATABASE	DROP_DATABASE
CREATE_ENDPOINT		DROP_ENDPOINT
CREATE_LOGIN	ALTER_LOGIN	DROP_LOGIN
GRANT_SERVER	DENY_SERVER	REVOKE_SERVER

- Using a database-scoped DDL trigger statements that can be captured are

CREATE_APPLICATION_ROLE	ALTER_APPLICATION_ROLE	DROP_APPLICATION_ROLE
CREATE_ASSEMBLY	ALTER_ASSEMBLY	DROP_ASSEMBLY
ALTER_AUTHORIZATION_DATABASE		
CREATE_CERTIFICATE	ALTER_CERTIFICATE	DROP_CERTIFICATE
CREATE_CONTRACT	DROP_CONTRACT	
GRANT_DATABASE	DENY_DATABASE	REVOKE_DATABASE
CREATE_EVENT_NOTIFICATION	DROP_EVENT_NOTIFICATION	
CREATE_FUNCTION	ALTER_FUNCTION	DROP_FUNCTION
CREATE_INDEX	ALTER_INDEX	DROP_INDEX
CREATE_MESSAGE_TYPE	ALTER_MESSAGE_TYPE	DROP_MESSAGE_TYPE
CREATE_PARTITION_FUNCTION	ALTER_PARTITION_FUNCTION	DROP_PARTITION_FUNCTION
CREATE_PARTITION_SCHEME	ALTER_PARTITION_SCHEME	DROP_PARTITION_SCHEME
CREATE_PROCEDURE	ALTER_PROCEDURE	DROP_PROCEDURE
CREATE_QUEUE	ALTER_QUEUE	DROP_QUEUE
CREATE_REMOTE_SERVICE_BINDING	ALTER_REMOTE_SERVICE_BINDING	DROP_REMOTE_SERVICE_BINDING
CREATE_ROLE	ALTER_ROLE	DROP_ROLE
CREATE_ROUTE	ALTER_ROUTE	DROP_ROUTE
CREATE_SCHEMA	ALTER_SCHEMA	DROP_SCHEMA
CREATE_SERVICE	ALTER_SERVICE	DROP_SERVICE
CREATE_STATISTICS	DROP_STATISTICS	UPDATE_STATISTICS
CREATE_SYNONYM	DROP_SYNONYM	
CREATE_TABLE	ALTER_TABLE	DROP_TABLE
CREATE_TRIGGER	ALTER_TRIGGER	DROP_TRIGGER
CREATE_TYPE	DROP_TYPE	
CREATE_USER	ALTER_USER	DROP_USER
CREATE_VIEW	ALTER_VIEW	DROP_VIEW
CREATE_XML_SCHEMA_COLLECTION	ALTER_XML_SCHEMA_COLLECTION	DROP_XML_SCHEMA_COLLECTION

Workout #07

- We want to log information about events - who, when creates or drops a table in a certain database.
- Whenever a table is created or dropped or altered, we want to store who, when did it and what SQL command is used.

i

value() method performs an XQuery against the XML and returns a value of SQL type. This method returns a scalar value. You typically use this method to extract a value from an XML instance stored in an xml type column, parameter, or variable. Syntax: value (XQuery, SQLType) XQuery is the XQuery expression, a string literal, that retrieves data inside the XML instance. The XQuery must return at most one value. SQLType is the preferred SQL type, a string literal, to be returned

- The information will be stored in a table.
- Create the table
CREATE TABLE ddl_log
(
PostTime datetime,
DB_User nvarchar(100),
Event nvarchar(100),
TSQL nvarchar(2000)

```

)
GO
--Create a DDL trigger
CREATE TRIGGER safety
ON DATABASE
FOR CREATE_TABLE , DROP_TABLE
AS
    DECLARE @data XML
    SET @data = EVENTDATA()

    INSERT INTO ddl_log
    (PostTime, DB_User, Event, TSQL)
VALUES
(
    GETDATE(),
    @data.VALUE('/EVENT_INSTANCE/LoginName')[1]',
    'NVARCHAR(100)',
    @data.VALUE('/EVENT_INSTANCE/EventType')[1]',
    'NVARCHAR(100)',
    @data.VALUE('/EVENT_INSTANCE/TSQLCommand')[1]',
    'NVARCHAR(2000)'
)
GO

```

The Trigger Code

- Let's break down the trigger code


```
CREATE TRIGGER safety
ON DATABASE
FOR CREATE_TABLE , DROP_TABLE
```
 - We're creating a trigger named safety that is scoped for the database


```
FOR CREATE_TABLE , DROP_TABLE
```
 - The trigger will fire when a create table, drop table statement is executed.


```
DECLARE @data xml
SET @data = EVENTDATA()

INSERT INTO ddl_log
(PostTime, DB_User, Event, TSQL)
VALUES
(GETDATE(),
CONVERT(nvarchar(100), CURRENT_USER),
@data.value('/EVENT_INSTANCE/EventType')[1]',
'nvarchar(100)',
@data.value('/EVENT_INSTANCE/TSQLCommand')[1]',
'nvarchar(2000)')
```
 - Remember that EVENTDATA will get the details of the executed DDL statement. We're simply assigning it to a local xml variable called @data.


```
INSERT INTO ddl_log
(PostTime, DB_User, Event, TSQL)
VALUES
(GETDATE(),
CONVERT(nvarchar(100), CURRENT_USER),
@data.value('/EVENT_INSTANCE/EventType')[1]',
'nvarchar(100)',
@data.value('/EVENT_INSTANCE/TSQLCommand')[1]',
'nvarchar(2000)')
```
- GETDATE() -returns current time,
 → CURRENT_USER - returns the user executing the statement and it is converted to nchar data
 → @data.value is calling the xquery function "value". The value function is defined as "value(Xquery, SQLType)".
 → @data.value('/EVENT_INSTANCE/EventType')[1]', 'nvarchar(100)') is going to return the full SQL statement that is executed to create or alter a table as a nvarchar(2000).

Workout #07 (Continued)

- Test the Trigger


```
CREATE TABLE NewTable (Column1 int);
GO
```
- See the log


```
SELECT * FROM ddl_log
GO
```

PostTime	DB_User	Event	TSQL
2012-12-25 21:55:27.787	IDB\Administrator	CREATE_TABLE	CREATE TABLE ..

- Drop the table


```
DROP TABLE NewTable
```

```

GO
See the log
SELECT * FROM ddl_log
GO

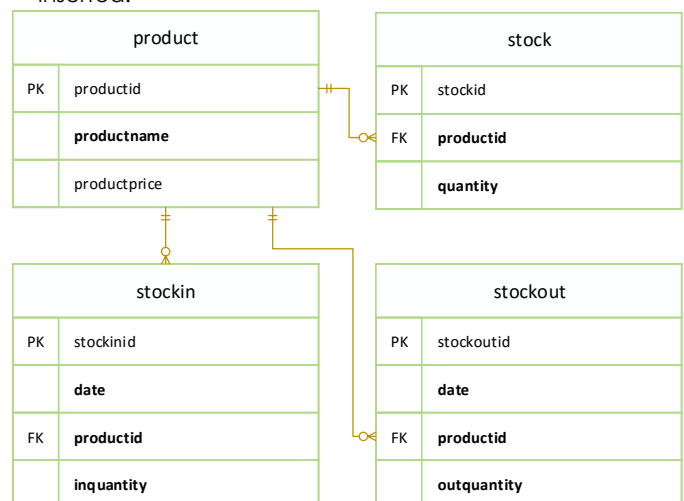
```

Result

PostTime	DB_User	Event	TSQL
2012-12-25 21:55:27.787	IDB\Administrator	CREATE_TABLE	CREATE TABLE ..
2012-12-25 21:58:34.423	IDB\Administrator	DROP_TABLE	DROP TABLE ..

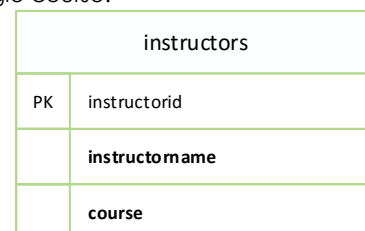
Practice #01

- Create the table with appropriate constraints and relationships as shown in the figure
- Create a after trigger on products table for inserts so that each time a new row is inserted into products table an entry is inserted in stocks with 0 value in stockquantity column
- Create an after trigger on stockin table for inserts so that each item a new row is inserted into stockin table, stockquantity column for corresponding product in stocks table is incremented by value in quantity value inserted.
- Create a after trigger on stockout table for inserts so that each item a new row is inserted into stockout table, stockquantity column for corresponding product in stocks table is decreased by value outquantity value inserted.



Practice #02

- Create the table as shown in the figure
- Create a trigger for insert action. The trigger will make sure that more than two instructors cannot be inserted for a single course.



Practice #03

- Create a table "laborworks" as shown below

Column name	Data Type	Allow Null
laborname	NVARCHAR(30)	No
datehired	DATETIME	No
enddate	DATETIME	Yes
wageperday	MONEY	No

- B. Create a view that shows data from laborworks as shown below

laborname	datehired	daysworked	wageperday	Payable
Mr. H	2012-12-02	3	90.00	270.00
..	..	..	..	..

- C. Create a instead of trigger on the view to insert data into the table "laborworks"

Practice #04

- A. You have a table ddl_log as described

Column	Data Type	Allow NULL
issuetime	DATETIME	No
db_user	NVARCHAR (100)	No
db_sql	NVARCHAR (100)	No

- B. Create a DDL trigger that monitors tables in a database. If any table created, altered or dropped an entry is created in the table ddl_log. Suppose a user creates a table, the trigger makes an entry in the ddl_log as shown

issuetime	db_user	db_sql
2012-12-29 10:30:16.360	dbo	CREATE TABLE TestTable (a int)

Practice #05

- A. Create two tables with relation as shown in the figure

Products table

Column name	Data Type	Allow Null	Key
ProductID	INT	No	Primary Key
ProductName	VARCHAR(30)	No	
StockLevel	INT	Yes	

Sales table

Column name	Data Type	Allow Null	Key
[Date]	DATETIME	No	
ProductID	INT	No	Foreign Key
Quantity	INT	No	

- B. Insert the following records in Stocks table

ProductID	ProductName	StockLevel
1	ATX Casing	210
2	DDR3 RAM 2G	130
3	CPU i3 3G	10

- C. Create an AFTER Trigger on Sales for INSERT action. The trigger deducts the Quantity value inserted in the Sales table from the Corresponding Product's StockLevel value in the Stocks table

Practice #06

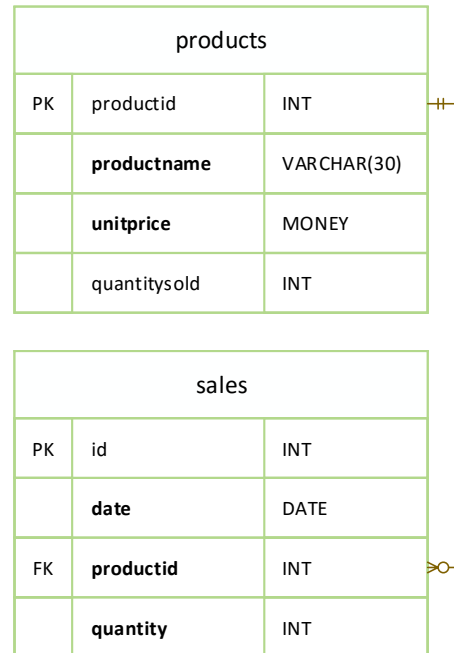
- A. Create a table "Classes" as Described

Column	Data type	Nullability
ClassId	INT	NOT NULL
Description	VARCHAR(100)	NOT NULL
ClassHour	INT	NOT NULL

- B. Create a trigger on "Classes" table so that an attempt to insert new row with ClassHour value less than 2 is rejected.
C. Insert some test values and show that the trigger is working.

Practice #07

- A. Create tables as shown in the figure



- B. Whenever a product is sold, the quantitysold column in products must be updated to store total quantity sold. Solve the problem using triggers.
C. Insert some test data in products table and add some sales record and verify that the trigger you created works fine.